Chapter 9: Objects and Classes

COMP 110
Examples, discussion, and practice

Learning goals

Model state and behavior with classes
Create objects through constructors
Explain references, aliases, and method calls
Use encapsulation, this, and static members
Store and process objects in arrays
Design immutable values

Discussion: modeling an object

A circle stores a radius and computes an area.
A rectangle stores two dimensions.
A button stores a label and can respond to a click.

Which information belongs to each object?
Which operations should its class provide?

An object you already use: String

```
String greeting = "Hello World";
System.out.println(greeting.length());
System.out.println(greeting.charAt(0));
```

The dot operator invokes a method through a reference.

Java: StringObjectDemo.java

Class, object, and reference

Class: Circle defines a kind of object.
Object: new Circle(2.0) creates one instance.
Reference: a variable can refer to that instance.

Two variables may refer to the same object.
Two objects can have different field values.

Circle: a class contract

Circle
- radius : double
+ Circle() + Circle(radius : double) + getRadius() : double + setRadius(radius : double) : void + getArea() : double + getPerimeter() : double

UML: - means private. + means public.

Java: Circle.java

Circle fields

```
private double radius;
private static int createdCount;
```

Each Circle has a radius. The class shares one construction count.

Java: Circle.java

Circle constructors

```
public Circle() {
    this(1.0);
}

public Circle(double radius) {
    validateRadius(radius);
    this.radius = radius;
    createdCount++;
}
```

new Circle() delegates to the radius constructor with 1.0.

Java: Circle.java

Getters, setters, and validation

```
public double getRadius() {
    return radius;
}

public void setRadius(double radius) {
    validateRadius(radius);
    this.radius = radius;
}

private static void validateRadius(double radius) {
    if (!Double.isFinite(radius) || radius < 0) {
        throw new IllegalArgumentException("Invalid radius");
    }
}
```

The setter preserves the finite, nonnegative radius rule.

Java: Circle.java

Computing behavior from current state

```
public double getArea() {
    return Math.PI * radius * radius;
}

public double getPerimeter() {
    return 2 * Math.PI * radius;
}
```

Area and perimeter use the current radius.

Java: Circle.java

A driver program creates and tests an object

```
Circle circle = new Circle(2.0);
System.out.println(circle.getRadius());
System.out.printf(java.util.Locale.US,
    "Area: %.2f%n", circle.getArea());
circle.setRadius(3.0);
System.out.println(circle.getRadius());
// circle.radius = -1; // Private field: does not compile.
```

Compile both Circle.java and TestCircle.java. Run TestCircle.

Java: TestCircle.java

Separate instances and shared references

```
Circle first = new Circle(2);
Circle second = new Circle(5);
Circle alias = first;
alias.setRadius(7);
System.out.println(first.getRadius());
System.out.println(second.getRadius());
System.out.println(first == alias);
System.out.println(first == second);
```

alias and first refer to one Circle. second refers to another.

Java: CircleAliasesDemo.java

Quiz 1: classes and constructors

1. Does new Circle(2) create a class or an object?
2. What is a constructor's return type?
3. Does Circle alias = first copy the object?
4. Can a driver directly assign a private radius?

When Java supplies a default constructor

```
private double radius = 1.0;

public double getRadius() {
    return radius;
}
```

No declared constructors means Java supplies a no-argument constructor.

Java: AutomaticCircle.java

An explicit constructor changes that rule

```
private double radius;

public RequiredRadius(double radius) {
    this.radius = radius;
}

public double getRadius() {
    return radius;
}
```

RequiredRadius(double) does not imply RequiredRadius().

Java: RequiredRadius.java

Testing constructor availability

```
AutomaticCircle automatic = new AutomaticCircle();
RequiredRadius explicit = new RequiredRadius(2);
System.out.println(automatic.getRadius());
System.out.println(explicit.getRadius());
// new RequiredRadius(); // No matching constructor.
```

The commented no-argument call has no matching constructor.

Java: ConstructorAvailabilityDemo.java

Default values of instance fields

```
int count;
boolean active;
String name;
```

int starts at 0, boolean at false, and a reference at null.

Java: DefaultFields.java

Default references and null access

```
DefaultFields item = new DefaultFields();
System.out.println(item.count);
System.out.println(item.active);
System.out.println(item.name);
Circle missing = null;
System.out.println(missing == null);
// missing.getArea(); // NullPointerException at run time.
```

null means the reference currently identifies no object.

Java: DefaultFieldsDemo.java

Primitive assignment and reference assignment

```
int a = 3;
int b = a;
b = 9;
Circle first = new Circle(3);
Circle second = first;
second.setRadius(9);
System.out.println(a);
System.out.println(first.getRadius());
```

Both assignments copy a value. The copied value has a different kind.

Java: PrimitiveAndReferenceDemo.java

Passing a primitive and an object reference

```
int number = 1;
Circle circle = new Circle(2);
change(number, circle);
System.out.println(number);
System.out.println(circle.getRadius());

static void change(int number, Circle circle) {
    number = 10;
    circle.setRadius(5);
}
```

The caller's int stays 1. Its Circle radius becomes 5.

Java: TestPassObject.java

Reassigning an object parameter

```
Circle circle = new Circle(2);
replace(circle);
System.out.println(circle.getRadius());

static void replace(Circle circle) {
    circle = new Circle(9);
}
```

The local parameter changes. The caller's reference stays the same.

Java: ReassignObjectDemo.java

Quiz 2: references and method calls

1. Can a setter called inside a method change a shared object?
2. Does reassigning the parameter replace the caller's reference variable?
3. Does Java ever pass these references by reference?

Discussion: encapsulation and valid state

Why keep radius private if a public setter exists?

Should every field have a setter?

What should a class do when a caller
requests an invalid value?

Rectangle: state and public operations

Rectangle
- length : double - width : double
+ Rectangle() + Rectangle(length : double, width : double) + getLength(), getWidth() : double + setLength(length : double) : void + setWidth(width : double) : void + getArea(), getPerimeter() : double

Store dimensions. Compute area and perimeter when requested.

Java: Rectangle.java

Rectangle fields and constructors

```
private double length;
private double width;

public Rectangle() {
    this(1.0, 1.0);
}

public Rectangle(double length, double width) {
    validate(length);
    validate(width);
    this.length = length;
    this.width = width;
}
```

The no-argument constructor delegates to a 1-by-1 rectangle.

Java: Rectangle.java

Rectangle accessors and mutators

```
public double getLength() { return length; }
public double getWidth() { return width; }

public void setLength(double length) {
    validate(length);
    this.length = length;
}

public void setWidth(double width) {
    validate(width);
    this.width = width;
}
```

this distinguishes a field from a parameter with the same name.

Java: Rectangle.java

A shared validation rule

```
private static void validate(double value) {
    if (!Double.isFinite(value) || value < 0) {
        throw new IllegalArgumentException("Invalid dimension");
    }
}
```

One private helper checks both dimensions.

Java: Rectangle.java

Derived values avoid stale area fields

```
public double getArea() {
    return length * width;
}

public double getPerimeter() {
    return 2 * (length + width);
}
```

Changing either dimension changes the next computed area.

Java: Rectangle.java

A Rectangle driver

```
Rectangle rectangle = new Rectangle(3, 4);
System.out.println(rectangle.getArea());
System.out.println(rectangle.getPerimeter());
rectangle.setWidth(5);
System.out.println(rectangle.getArea());
```

The updated area is 15.0 after width becomes 5.0.

Java: TestRectangle.java

Field scope, local scope, and this

```
private int value = 2;

void show(int value) {
    int doubled = value * 2;
    System.out.println(value);
    System.out.println(this.value);
    System.out.println(doubled);
}
```

The parameter value shadows the field. this.value still names the field.

Java: FieldScopeDemo.java

Static state and instance state

```
public static int getCreatedCount() {
    return createdCount;
}
```

Call Circle.getCreatedCount() to read class-wide state.

Java: Circle.java

One shared count, separate radii

```
int before = Circle.getCreatedCount();
Circle first = new Circle();
Circle second = new Circle(2);
System.out.println(Circle.getCreatedCount() - before);
first.setRadius(4);
System.out.println(first.getRadius());
System.out.println(second.getRadius());
```

ClassName.member makes static access clear.

Java: StaticCircleDemo.java

Quiz 3: this and static members

1. What does this.radius identify?
2. Can a static method use this?
3. Is the creation count separate for each Circle?
4. Does the count tell us how many Circles currently remain reachable?

Library objects: Date

```
Date date = new Date(0L);
System.out.println(date.getTime());
date.setTime(1000L);
System.out.println(date.getTime());
```

getTime returns milliseconds since the Unix epoch.

Java: DateObjectDemo.java

Library objects: Random

```
Random first = new Random(9);
Random second = new Random(9);
for (int i = 0; i < 3; i++) {
    int a = first.nextInt(10);
    int b = second.nextInt(10);
    System.out.println(a + " " + b);
}
```

The same seed and calls produce the same pseudorandom sequence.

Java: RandomObjectDemo.java

Library objects: a two-dimensional point

```
Point2D point = new Point2D.Double(3, 4);
System.out.println(point.getX());
System.out.println(point.getY());
System.out.println(point.distance(0, 0));
point.setLocation(6, 8);
System.out.println(point.distance(0, 0));
```

This example uses java.awt.geom.Point2D from standard Java SE.

Java: PointObjectDemo.java

Discussion: reading an unfamiliar class API

Which constructor creates the object?
Which methods read state?
Which methods change state?
What types do the methods return?

How can a small driver test your understanding?

An array of references needs object initialization

```
Circle[] circles = new Circle[3];
System.out.println(circles[0] == null);
for (int i = 0; i < circles.length; i++) {
    circles[i] = new Circle(i + 1);
}
for (Circle circle : circles) {
    System.out.println(circle.getRadius());
}
```

new Circle[3] creates three null reference slots, not three Circles.

Java: CircleArrayDemo.java

Case study: areas of five random Circles

```
Random random = new Random(9);
Circle[] circles = new Circle[5];
for (int i = 0; i < circles.length; i++) {
    circles[i] = new Circle(1 + random.nextInt(5));
}
for (Circle circle : circles) {
    System.out.printf(java.util.Locale.US, "r=%.0f area=%.2f%n",
        circle.getRadius(), circle.getArea());
}
System.out.printf(java.util.Locale.US, "Total: %.2f%n",
    totalArea(circles));
```

Fixed seed, initialized objects, per-object output, and a total.

Java: TotalArea.java

Processing objects through their public interface

```
static double totalArea(Circle[] circles) {
    double total = 0;
    for (Circle circle : circles) total += circle.getArea();
    return total;
}
```

The total method asks each Circle for its area.

Java: TotalArea.java

An array copy can still share its objects

```
Circle[] original = {new Circle(2), new Circle(3)};
Circle[] shallow = original.clone();
shallow[0].setRadius(9);
System.out.println(original[0].getRadius());
Circle[] independent = copyCircles(original);
independent[0].setRadius(4);
System.out.println(original[0].getRadius());
System.out.println(independent[0].getRadius());
```

clone copies the Circle references into a new array.

Java: CopyCircleArrayDemo.java

Copying the Circle objects too

```
static Circle[] copyCircles(Circle[] source) {
    Circle[] copy = new Circle[source.length];
    for (int i = 0; i < source.length; i++) {
        copy[i] = new Circle(source[i].getRadius());
    }
    return copy;
}
```

New model objects give independent radius state.

Java: CopyCircleArrayDemo.java

Quiz 4: arrays of objects

1. What does a new Circle[5] contain?
2. Does cloning that array clone the Circles?
3. Can a for-each loop call a setter on
   a Circle stored in the array?
4. What must totalArea assume about its elements?

Designing an immutable value

Set all state during construction.
Keep fields private and prevent reassignment.
Provide no state-changing operations.
Protect mutable objects held inside the value.

A final reference can still identify a mutable object.

An immutable point with primitive state

```
private final int x;
private final int y;

public ImmutablePoint(int x, int y) {
    this.x = x;
    this.y = y;
}

public int getX() { return x; }
public int getY() { return y; }
```

The constructor sets the coordinates. Getters expose their values.

Java: ImmutablePoint.java

A new immutable value leaves the old one intact

```
ImmutablePoint point = new ImmutablePoint(3, 4);
ImmutablePoint alias = point;
point = new ImmutablePoint(6, 8);
System.out.println(alias.getX() + "," + alias.getY());
System.out.println(point.getX() + "," + point.getY());
```

The old reference still sees coordinates (3, 4).

Java: ImmutablePointDemo.java

Protecting mutable data inside an immutable class

```
private final int[] values;

public ImmutableScores(int[] values) {
    this.values = values.clone();
}

public int[] getValues() {
    return values.clone();
}
```

Copy the int array when accepting it and when returning it.

Java: ImmutableScores.java

Testing defensive copies

```
int[] input = {80, 90};
ImmutableScores scores = new ImmutableScores(input);
input[0] = 0;
int[] returned = scores.getValues();
returned[1] = 0;
System.out.println(Arrays.toString(scores.getValues()));
```

Changes through both outside arrays leave the stored scores intact.

Java: ImmutableScoresDemo.java

Case study: Rational numbers

Represent an exact fraction with two integers.
Reduce it to lowest terms.
Keep the denominator positive and nonzero.
Represent zero as 0/1.
Return new values from arithmetic methods.

Rational: an immutable interface

Rational
- numerator : int, final - denominator : int, final
+ Rational() + Rational(n : int, d : int) + add(), subtract() : Rational + multiply(), divide() : Rational + compareTo(other : Rational) : int + equals(object : Object) : boolean + toString() : String

Arithmetic receives another Rational and returns a new Rational.

Java: Rational.java

Rational fields and public constructors

```
private final int numerator;
private final int denominator;

public Rational() {
    this(0, 1);
}

public Rational(int numerator, int denominator) {
    this((long) numerator, (long) denominator);
}
```

The complete class declaration is public final class Rational.

Java: Rational.java

Normalizing a Rational value

```
private Rational(long n, long d) {
    if (d == 0) {
        throw new IllegalArgumentException("Zero denominator");
    }
    if (d < 0) { n = -n; d = -d; }
    long divisor = gcd(Math.abs(n), d);
    this.numerator = Math.toIntExact(n / divisor);
    this.denominator = Math.toIntExact(d / divisor);
}
```

Example: 6/-8 becomes -3/4. A zero denominator is invalid.

Java: Rational.java

A private greatest-common-divisor helper

```
private static long gcd(long a, long b) {
    while (b != 0) {
        long remainder = a % b;
        a = b;
        b = remainder;
    }
    return a;
}
```

The Euclidean algorithm supports fraction reduction.

Java: Rational.java

Accessors and readable output

```
public int getNumerator() { return numerator; }
public int getDenominator() { return denominator; }

@Override
public String toString() {
    return numerator + "/" + denominator;
}
```

toString always formats the canonical numerator/denominator pair.

Java: Rational.java

Adding and subtracting exact fractions

```
public Rational add(Rational other) {
    long n = (long) numerator * other.denominator
           + (long) other.numerator * denominator;
    long d = (long) denominator * other.denominator;
    return new Rational(n, d);
}

public Rational subtract(Rational other) {
    long n = (long) numerator * other.denominator
           - (long) other.numerator * denominator;
    long d = (long) denominator * other.denominator;
    return new Rational(n, d);
}
```

Cross-multiply, then let the constructor reduce the result.

Java: Rational.java

Multiplication and division

```
public Rational multiply(Rational other) {
    return new Rational((long) numerator * other.numerator,
        (long) denominator * other.denominator);
}

public Rational divide(Rational other) {
    if (other.numerator == 0) {
        throw new ArithmeticException("Division by zero");
    }
    return new Rational((long) numerator * other.denominator,
        (long) denominator * other.numerator);
}
```

A division by a zero Rational raises an exception.

Java: Rational.java

Comparing Rational values

```
public int compareTo(Rational other) {
    long left = (long) numerator * other.denominator;
    long right = (long) other.numerator * denominator;
    return Long.compare(left, right);
}
```

Cross products compare exact values without floating-point conversion.

Java: Rational.java

Value equality and hash codes

```
@Override
public boolean equals(Object object) {
    if (!(object instanceof Rational)) return false;
    Rational other = (Rational) object;
    return numerator == other.numerator
        && denominator == other.denominator;
}

@Override
public int hashCode() {
    return 31 * numerator + denominator;
}
```

Canonical forms let equality compare the two stored fields.

Java: Rational.java

Testing Rational arithmetic

```
Rational first = new Rational(1, 2);
Rational second = new Rational(1, 3);
System.out.println(first.add(second));
System.out.println(first.subtract(second));
System.out.println(first.multiply(second));
System.out.println(first.divide(second));
System.out.println(first);
System.out.println(first.equals(new Rational(2, 4)));
```

The final first output remains 1/2 after all operations.

Java: RationalDriver.java

Case study: the Rational summation series

```
Scanner input = new Scanner(System.in);
int n = input.nextInt();
if (n < 1) {
    throw new IllegalArgumentException("n must be positive");
}
Rational sum = new Rational();
StringBuilder expression = new StringBuilder();
for (int i = 1; i <= n; i++) {
    if (i > 1) expression.append(" + ");
    Rational term = new Rational(1, i);
    expression.append(term);
    sum = sum.add(term);
}
System.out.println(expression + " = " + sum);
```

For n = 3: 1/1 + 1/2 + 1/3 = 11/6

Java: SumRational.java

Case study: the maximum Rational

```
Scanner input = new Scanner(System.in);
int count = input.nextInt();
if (count < 1) {
    throw new IllegalArgumentException("Empty input");
}
Rational best = new Rational(input.nextInt(), input.nextInt());
for (int i = 1; i < count; i++) {
    Rational value = new Rational(
        input.nextInt(), input.nextInt());
    if (value.compareTo(best) > 0) best = value;
}
System.out.println(best);
```

Input: 3 1 2 3 4 2 3. Output: 3/4.

Java: MaxRational.java

Quiz 5: immutable Rational values

1. What canonical value represents 6/-8?
2. Can a denominator be zero?
3. Does first.add(second) change first?
4. Should two separately constructed 1/2 values necessarily satisfy first == second?
5. What does compareTo returning 0 mean?

Exit exercise: a Counter class

Design a Counter with:
A private count initially equal to zero
An increment() operation
A reset() operation
A getCount() accessor

What should remain outside its public interface?

Exit exercise: one possible Counter

```
private int count;

public void increment() {
    count = Math.incrementExact(count);
}

public void reset() {
    count = 0;
}

public int getCount() {
    return count;
}
```

Named operations preserve control over private state.

Java: Counter.java

Testing the Counter interface

```
Counter counter = new Counter();
counter.increment();
counter.increment();
System.out.println(counter.getCount());
counter.reset();
System.out.println(counter.getCount());
```

Two increments give 2. A reset gives 0.

Java: CounterDemo.java

Chapter review

Constructors establish object state.
References can share one mutable object.
Methods receive copied argument values.
Private fields support controlled public operations.
Static members belong to the class.
Immutable values need protected internal state.